

How to get a new instance of an exported object on each import?

Asked 3 years, 10 months ago Modified 3 years, 10 months ago Viewed 1k times



I have this file that I'm keeping a INITIAL_VALUE for a form field, that I'm building.

3

INITIAL_VALUE.js



```
const INITIAL_VALUE = [];  
  
export default INITIAL_VALUE;
```



And the problem is that INITIAL_VALUE is an array. A non-primitive, that is handled by reference.

Component1.js

```
import INITIAL_VALUE from "../INITIAL_VALUE";  
import React, { useState } from "react";  
  
function Component1(props) {  
  const [myState, setMyState] = useState(INITIAL_VALUE);  
  const [boolean, setBoolean] = useState(false);  
  
  function handleClick() {  
    setMyState(prevState => {  
      prevState.push(1);  
      return prevState;  
    });  
    setBoolean(prevState => !prevState);  
    props.forceUpdateApp();  
  }  
  
  return (  
    <React.Fragment>  
      <div>This is my state: {JSON.stringify(myState)}</div>  
      <button onClick={handleClick}>Modify State Comp1</button>  
    </React.Fragment>  
  );  
}  
  
export default Component1;
```

Component2.js

The same as Component1, but it's named Component2 and it has its own file.

App.js

```
function App() {  
  const [boolean, setBoolean] = useState(false);
```

```
function forceUpdateApp() {
  setBoolean(prevState => !prevState);
}

return (
  <React.Fragment>
    <Component1 forceUpdateApp={forceUpdateApp} />
    <Component1 forceUpdateApp={forceUpdateApp} />
    <Component2 forceUpdateApp={forceUpdateApp} />
  </React.Fragment>
);
}
```

[CodeSandbox](#)

PROBLEM

`Component1.js` and `Component2.js` both import the `INITIAL_VALUE` file. And **I was under the impression that, each one of these imports would get a brand new instance of the `INITIAL_VALUE` object**. But that is not the case as we can see from the GIF below:

 [enter image description here](#)

QUESTION

Is there a way to keep an array as a initial value living declared and imported from another file and always **get a new reference to it on each import**? Is there another pattern I can use to solve this? Or should I stick with only primitive values and make it `null` instead of `[]` and initialize it in the consumer file?

javascript

arrays

reactjs

object

primitive

Share Improve this question Follow

edited Aug 1, 2019 at 7:37

asked Aug 1, 2019 at 7:22



cbdeveloper

27.3k ● 34 ● 149 ● 314

1 Answer

Sorted by:

Highest score (default)



4

Is there a way to keep an array as a initial value living declared and imported from another file and always get a new reference to it on each import?



No, that's not possible. The top-most level code of a module will run *once*, at most. Here, the top level of `INITIAL_VALUE.js` defines *one* array and exports it, so everything that imports it will have



a reference to that same array.



Easiest tweak would be to export a function which creates the array instead:



```
// makeInitialValue.js
export default () => {
  const INITIAL_VALUE = [];
  // the created array / object can be much more complicated, if you wish
  return INITIAL_VALUE;
};
```

and then

```
import makeInitialValue from "./makeInitialValue";
function Component1(props) {
  const INITIAL_VALUE = makeInitialValue();
  const [myState, setMyState] = useState(INITIAL_VALUE);
```

In the simplified case that you just need an empty array, it would be easier just to define it when you pass it to `useState`.

All that said, it would be much better to fix your code so that it does not mutate the existing state. Change

```
setMyState(prevState => {
  prevState.push(1);
  return prevState;
});
```

to

```
setMyState(prevState => {
  return [...prevState, 1];
});
```

That way, even if all components and component instances start out with the same array, it won't cause problems, because the array will never be mutated.

Share Improve this answer Follow

edited Aug 1, 2019 at 7:34

answered Aug 1, 2019 at 7:26



CertainPerformance

353k ● 51 ● 301 ● 313

Thanks for your answer. I just tried that, but what happens is that `Component2` gets a new instance. But both `Component1` end up sharing the same instance. Is there a solution to this? – [cbdeveloper](#) Aug 1, 2019 at 7:28

There is no need to invoke `INITIAL_VALUE` function, it can be passed as-is to `useState` and it will be lazily initialized: reactjs.org/docs/hooks-reference.html#lazy-initial-state – [haim770](#) Aug 1, 2019 at 7:29 

My real case I have like 5 possible initial values that are based on some form field `type`. Some of the types should have empty `array`, other will have empty `string`, others are `false`, etc. – [cbdeveloper](#) Aug 1, 2019 at 7:30

My bad, I was invoking the function at top-level. If I invoke it inside the component is all good and every one gets its own instance! Thanks!! – [cbdeveloper](#) Aug 1, 2019 at 7:31

@cbdev420 Keep in mind that this is a *bit* of an X/Y problem, for this particular situation with React - it would be better never to mutate the existing state at all. Better to create a new array every time `setMyState` is called – [CertainPerformance](#) Aug 1, 2019 at 7:35 
